

LUMORA SPACE

FULL STACK DEVELOPMENT

MERN Stack Development with GenAI

*A twelve-week path from your first commit to a portfolio
you'd actually want a hiring manager to open, built around
the way modern product teams write, review, and ship code.*

12

WEEKS, LIVE

60+12

WEEKDAY + CAREER
SESSIONS

11+1

MILESTONE PROJECTS +
CAPSTONE

8+

AI TOOLS USED ON THE
JOB

From confused to confident.

Most students don't fail at learning to code. They fail at figuring out what to learn next, in what order, and why it matters. They find that out alone, at 1 a.m., twelve tabs deep in a tutorial that promised to make them "job-ready" in thirty days.

LumoraSpace exists to close that gap. We build curricula the way a hiring manager would design them if they had the time: starting from what a junior developer actually does in their first ninety days on a real team, and working backward to a week-by-week plan that gets a student there.

That means every curriculum we run, including this one, goes through the same discipline: we research how the best programs in the world teach this subject, benchmark our own draft against them, find the gaps, and rebuild before a single student sees it. Nothing here is guesswork. It's the result of comparing this course against the leading full-stack programs, and fixing what they leave out.

We don't teach students to finish a course. We teach them to be trusted with a codebase.

This brochure is about one program, MERN Stack Development with GenAI, but it reflects a philosophy that runs through everything we build: practical over theoretical, shipped over studied, and reviewed over assumed correct.

Why most students stay stuck.

None of this is about talent. It's about structure.

01 Tutorial hell

Every new tutorial feels like progress. Most of them teach the same beginner concepts in a different accent, and never actually get harder.

02 No real roadmap

Should you learn TypeScript before or after React? Redux or TanStack Query? Nobody tells you the order, so you either skip fundamentals or drown in them.

03 No mentorship or feedback

You can watch a thousand videos and still not know if the code you just wrote is any good. There's no one to tell you what a real team would flag in review.

04 No accountability

Self-paced courses have a 90%+ drop-off rate for a reason. Without a cohort, a deadline, and someone checking your work, momentum dies quietly.

05 No real projects, or too many of the same one

Every portfolio has a to-do app. Recruiters have seen it a thousand times. Building something is not the same as building something that proves you can think.

06 No interview preparation

Knowing React is not the same as explaining a technical decision under pressure to someone deciding whether to hire you.

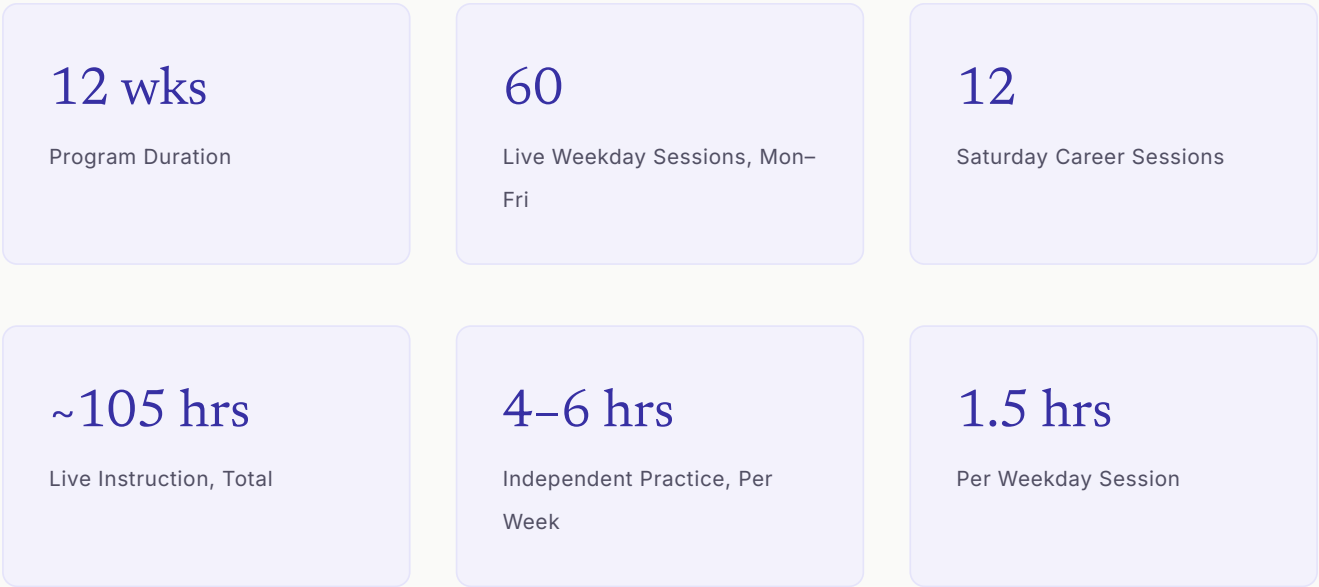
07 No clarity on where AI fits

Some courses ignore AI tools entirely. Others let students lean on them so heavily they can't explain their own code in an interview. Neither prepares you for a real job.

LumoraSpace was built specifically against this list: sequencing, mentorship, accountability, original projects, interview preparation, and a disciplined approach to AI, in one twelve-week program.

ABOUT THE PROGRAM

Twelve weeks. One structure. No wasted sessions.



Every weekday session runs live, and follows the same internal rhythm, deliberately weighted away from lecture and toward building, because that's where retention actually happens.

SHARE	PHASE	WHAT HAPPENS
20%	Core Concepts	Only the theory needed to understand today's build, with no detours into concepts you won't use for weeks.
30%	Live Demonstration	The instructor builds the day's feature live, narrating every decision, including the ones that don't work on the first try.
50%	Hands-On Practice	Students build their own version, break it, debug it, and extend it, with support in the room.

Saturdays are reserved for career sessions: resume work, GitHub and LinkedIn reviews, mock interviews, and system-design basics. Career preparation happens in parallel with the curriculum, not as an afterthought in the final week.

Compared to how most people actually learn to code.

What Matters	Free Tutorials	A Traditional CS Course	A Recorded Bootcamp	LumoraSpace
Sequencing	Whatever the algorithm recommends next	Theory-first, paced by a semester calendar	Fixed curriculum, same for every student	Every week's build requires the exact skill taught the week before
Feedback	None. You're on your own	Graded assignments, rarely a real code review	A community forum, best effort	Code reviews, peer reviews, and 1:1 portfolio review, built into the calendar
AI in the workflow	A shortcut to skip understanding, not build it	Rarely addressed directly	Often a bolted-on extra module	A verification discipline threaded through all 12 weeks: "AI drafts, you're responsible for the answer"
Projects	Copy-along, rarely deployed	Academic, rarely shipped or public on GitHub	Follow-along, sometimes original	11 original projects + a capstone, every one deployed, documented, and on GitHub
Career preparation	None	A general placement cell, not role-specific	Templates and generic advice	Mock interviews, resume/ LinkedIn/ GitHub review, and a role-specific post-course roadmap

None of this is a claim that other paths can't work. Plenty of great engineers are self-taught. It's a claim about what most people are missing when they try to do it alone, and what a structured, mentored, live cohort adds back in.

Twelve weeks. Twelve transformations.

Not what you'll study, but what you'll be able to do by Friday of each week.

Week 01	HTML, CSS, Tailwind & Git	<i>You go from a blank page to a live, responsive site, and a Git habit that never breaks.</i>
Week 02	JavaScript & the DOM	<i>JavaScript stops being an abstract language and starts being the thing that makes your pages respond to a real person.</i>
Week 03	React Fundamentals	<i>You build your first real interface: components, state, interaction. No backend behind it yet, and none needed.</i>
Week 04	TypeScript, Routing & Code Review	<i>You add types, add real navigation, and do something most bootcamps skip entirely: read someone else's code and open a pull request.</i>
Week 05	Node.js & Express	<i>You build a backend with no UI in sight, and learn to trust it because you tested every endpoint yourself.</i>
Week 06	MongoDB, Mongoose & Validation	<i>Your API gets a real database behind it: validated input, indexed queries, and answers to real business questions.</i>
Week 07	Full-Stack Integration	<i>Frontend meets backend for the first time, in something with the shape of a real product, not another to-do list.</i>
Week 08	Auth, Roles & Security	<i>Login, roles, file uploads, and the security checklist a real team runs before anything ships.</i>

Continued on the next page: Weeks 09 through 12.

Weeks nine through twelve.

Week 09	Advanced State & Debugging	<i>State gets genuinely complicated, and you learn to manage it, and to debug without reaching for AI as your first move.</i>
Week 10	Next.js Migration	<i>You migrate a real app to Next.js and finally understand, hands-on, what runs on the server versus what runs in the browser.</i>
Week 11	Testing, Docker & CI/CD	<i>You test it, harden it, containerize it, and push it through a CI/CD pipeline, like an engineer joining a team, not a student finishing a course.</i>
Week 12	GenAI Integration & Capstone	<i>You ship a capstone with a real AI-powered feature in it, and you can explain, line by line, exactly why it works.</i>

By the time you present your capstone, "I followed a tutorial" isn't a sentence you'll need anymore.

Every single week ends with a project pushed to GitHub, never a slide deck, never a quiz score standing in for proof of skill. Eleven milestone projects plus one capstone, and no two of them are the same kind of application.

The full stack, in the order that makes sense.

LAYER	WEEKS	WHAT'S COVERED
Frontend	1–4, 7, 9	Semantic HTML, Tailwind CSS, JavaScript (ES6+), React, TypeScript, React Router, controlled forms, Redux Toolkit for complex state
Backend	5, 7–9	Node.js, Express, RESTful API design, middleware, error handling, clean architecture
Database	6	MongoDB, Mongoose schema design, indexing, the aggregation pipeline, Zod validation
Authentication	8	JWT, bcrypt password hashing, role-based access control, OAuth concepts, managed auth providers
Next.js	10	App Router, Server and Client Components, route handlers, migrating a classic MERN frontend
Testing & DevOps	11	Vitest, Testing Library, Playwright (guided), reading and modifying a Dockerfile, extending a GitHub Actions pipeline
Deployment	7, 10–12	Vercel (frontend/Next.js), Railway/Render or container-based deploys (backend)
GenAI Integration	1–12 (threaded), 12 (dedicated)	AI-assisted development throughout; a dedicated unit on building a real LLM-powered feature with streaming responses and structured output validation

Nothing here is taught in isolation. Each layer connects to a real project the same week it's introduced. You never learn a concept without immediately using it to build something that works.

What you'll actually be able to put on a resume.

Frontend

Semantic HTML5 · Flexbox & CSS Grid · Tailwind CSS · Modern JavaScript (ES6+) · TypeScript · React (hooks, composition, controlled forms) · React Router · Next.js App Router · Server & Client Components · Redux Toolkit · TanStack Query · React Hook Form + Zod

Backend

Node.js runtime & module system · Express routing & middleware · RESTful API design · error handling · JWT authentication · bcrypt · role-based access control · Socket.IO (real-time, optional)

Database

MongoDB & MongoDB Compass · Mongoose schema design · embedding vs. referencing · indexing · aggregation pipelines · Zod-based validation

Cloud & Deployment

Vercel · Railway / Render · Cloudinary file uploads · Docker (reading & modifying) · GitHub Actions CI/CD (extending an existing pipeline)

Version Control

Git fundamentals · clean commit history · branching · the Pull Request workflow · giving and receiving code review feedback

AI Tools

ChatGPT & Claude for concept explanation, debugging, and documentation · GitHub Copilot & Cursor for reviewed code generation · v0, Bolt.new & Lovable for rapid prototyping · OpenAI, Anthropic & Gemini APIs for real feature integration

Professional Skills

Reading an unfamiliar codebase · writing a clear Pull Request description · structured logging & debugging without AI · clean-code principles · project documentation · explaining a technical decision under interview conditions

Testing & Quality

Vitest & Testing Library (unit/integration) · Playwright (guided E2E exposure) · rate limiting & API versioning · frontend performance basics · error boundaries

AI accelerates developers. It doesn't replace them.

"AI writes the first draft. You are responsible for the final answer."

That single sentence is taught in Week 1 and reinforced in every session after it. Students use ChatGPT, Claude, GitHub Copilot, Cursor, Gemini, v0, Bolt.new, and Lovable across the entire development lifecycle, but never as a way to skip understanding.

WHERE AI IS USED	HOW STUDENTS ARE TRAINED TO USE IT
Learning a new concept	Ask for a second explanation when a lesson didn't click, then confirm understanding by applying it in code, not by re-reading the answer.
Code generation	Use for boilerplate and autocomplete, but every line must be explained before it's committed.
Debugging	Paste the real error and context, get a diagnosis, then verify the actual cause in the code before applying any fix.
UI generation (v0 / Bolt / Lovable)	Used for rapid prototyping, never shipped without being read, understood, and often rewritten.
Documentation & PR descriptions	AI drafts structure and phrasing; the student supplies the real, project-specific facts.

Responsible use, taught explicitly:

- Never commit AI-generated code without running and understanding every line
- Never paste real secrets, API keys, or personal data into any AI tool
- Always verify AI-suggested database queries against known-answer sample data
- Practice debugging without AI periodically, in a formal, AI-free session built into Week 9
- Document exactly where and how AI was used in every project's docs/ai-workflow.md

Eleven projects. No two the same.

Every project ships: deployed, documented, and pushed to a GitHub repository built to the standard hiring managers expect.

01 · Responsive Landing Site

MINI PROJECT

A fully responsive, Tailwind-styled static site, version-controlled from the very first commit.

Skills: Semantic HTML · Flexbox/Grid · Responsive design · Tailwind · Git

02 · Interactive DOM-Based App

MINI PROJECT

Proof of real JavaScript fluency, a genuinely interactive browser app with no framework behind it.

Skills: ES6+ syntax · Array/object methods · Async/await · DOM manipulation

03 · Client-Only Interactive App

MINI PROJECT

A deployed React app built around a distinctive UX pattern, an onboarding wizard or drag-and-drop board, never a generic list-and-filter app.

Skills: React components · Hooks · Controlled forms · Component composition

04 · TypeScript-Migrated App + Reviewed PR

FEATURE PROJECT

A type-safe, multi-route app, plus a real, reviewed Pull Request against an unfamiliar codebase.

Skills: TypeScript · React Router · Codebase navigation · PR writing · Code review etiquette

05 · Standalone REST API

MODULE PROJECT

A documented, tested backend built and verified with no frontend in sight.

Skills: Node.js · Express · REST API design

06 · Database-Backed API

MODULE PROJECT

The Week 5 API, extended with a real, validated, indexed database layer.

Skills: MongoDB · Mongoose · Indexing · Aggregation · Zod · Clean architecture

Continued on the next page: Projects 07 through the Capstone.

Where it becomes a real product.

07 · Full-Stack Booking / Marketplace App

MAJOR PROJECT

Frontend connected to a live backend with genuine product complexity: booking conflict detection or marketplace search and filtering.

Skills: Axios · TanStack Query · React Hook Form + Zod

08 · Authenticated App Extension

MAJOR PROJECT (EXTENDED)

The Week 7 app, secured with production-realistic auth, roles, and file uploads.

Skills: JWT · bcrypt · RBAC · Cloudinary · Security hardening

09 · State-Heavy App (Real-Time Optional)

MAJOR PROJECT

Complex client state managed at production depth, with logging discipline and an optional real-time layer.

Skills: Redux Toolkit · RTK Query · Structured logging · Socket.IO (optional)

10 · Next.js Migration

MAJOR PROJECT

A classic MERN frontend, migrated to Next.js App Router and deployed on Vercel.

Skills: Next.js · Server/Client Components · Route handlers

11 · Production-Hardened App

MAJOR PROJECT (HARDENED)

A prior major project, tested, hardened, containerized, and shipped through a working CI/CD pipeline.

Skills: Vitest · Testing Library · Rate limiting · Docker (read/modify) · GitHub Actions

12 · Capstone

FLAGSHIP PROJECT

A full production-shaped application with a real GenAI-powered feature, incorporating real code review feedback and presented live to a panel.

Skills: Full stack + GenAI API integration + structured output validation + review response

Every README includes: the problem statement, tech stack, key features, screenshots or a demo, a live link, and an "AI Usage & Verification" section documenting exactly how AI tools were used and checked.

One week. One real product. One AI feature you can defend.

Week 12 isn't a review week. It's a full build sprint. Students choose their own real-world problem, scope it to something achievable in five days, and ship a production-shaped application with a genuine GenAI-powered feature: a chat assistant, smart search, or summarization tool, built on the OpenAI, Anthropic, or Gemini APIs.

DAY	WHAT HAPPENS
Monday	Capstone kickoff: choosing a real idea and scoping a one-week, production-shaped plan
Tuesday	GenAI API fundamentals: server-side calls, API key security, streaming responses to the frontend
Wednesday	Structured output validation with Zod: handling unpredictable AI output safely, with graceful fallbacks
Thursday	Full build day, with instructor check-ins throughout
Friday	Documentation, GitHub polish, and a live presentation to a panel, with technical Q&A

Why this matters to an employer:

Any student can claim they "used AI to build a project." Few can walk a panel through exactly where an LLM call happens, how its output is validated before it reaches a user, and which line of the README documents a decision they changed after real code review feedback. That's what the capstone evaluates, and it's what a technical interviewer actually wants to see.

Built in from Week 1, not bolted on in Week 12.

Resume

Built and refined across dedicated Saturday sessions plus a Week 11 pass: project-driven, metrics-backed, and ATS-aware.

GitHub Portfolio

Eleven milestone projects plus the capstone, all following one repository standard, reviewed at Weeks 4 and 11.

LinkedIn

A full profile buildout (headline, About section, featured projects, and skills) with a dedicated Saturday session.

Resume + Portfolio Review

A 1:1 or small-group review measured against real hiring-manager expectations.

System Design Basics

Reading and reasoning about a simple system diagram, and what "scalability" actually means at entry level.

Networking & Referrals

How to reach out to alumni and recruiters on LinkedIn without sounding like everyone else who just finished a bootcamp.

Interview preparation, round by round:

JavaScript / React Technical	Saturday, Week 7	Timed live-coding on fundamentals, React patterns, and debugging
Backend & System Design Basics	Saturday, Week 8	Timed questions on REST API design, data modeling, and basic system design
HR / Behavioral	Saturday, Week 10	"Tell me about yourself," project walkthroughs, behavioral questions
Full Mock Interview	Saturday, Week 11	Complete live-coding + system design + behavioral round, recorded, with feedback
Final Capstone Presentation	Saturday, Week 12	Live presentation to a panel, with technical Q&A

Feedback from people who've actually shipped code.

A tutorial can't tell you your code is fragile. A mentor can, and does, on a schedule.

Structured code reviews

Formal reviews at Weeks 4, 7, 9, and 11 evaluate real project code against professional standards, not just whether it runs, but whether a team would accept it.

Peer review practice

At Weeks 4 and 9, students review each other's Pull Requests with specific, constructive feedback, the exact skill used on every real engineering team.

1:1 portfolio and resume review

A dedicated resume and portfolio session measured against what hiring managers are actually screening for at entry level.

A panel at the finish line

The capstone is presented live to a panel (instructor, peers, and mentors or hiring-manager guests where available) with real technical questions, not a formality.

The goal of mentorship here isn't encouragement for its own sake. It's the same kind of direct, specific feedback a junior engineer would get from a senior teammate, early enough in the course that there's time to actually act on it.

You're building alongside people, not just a syllabus.

A cohort, not a solo course

Every session is live and shared with the same group of students for twelve weeks, the same accountability that makes a gym partner more effective than a fitness app.

A peer-review culture

Reviewing a classmate's Pull Request twice during the course means you leave knowing how to give and receive feedback on real code, not just write it.

Hackathons, after the fundamentals are solid

Guidance on entering beginner-friendly full-stack hackathons once the core skills are in place, a natural next step for students who want to keep building at pace.

Open source and freelancing tracks

Bonus guidance on contributing to beginner-friendly open-source MERN/Next.js projects, and on positioning full-stack skills for small business or MVP-building freelance work.

The people you build alongside for twelve weeks are often the same people who'll send you a job posting, review your code six months from now, or be your first professional reference.

Built for the students most bootcamps overlook.

Engineering students who know some code, but not what comes next

You've written functions and maybe a small project. You don't have a clear picture of how those skills become a job.

Freshers and beginners with zero prior experience

No prior experience with JavaScript, React, Node.js, or Git is required. Every concept is built from the ground up.

Students from Tier 2 and Tier 3 colleges

Where campus placement support and mentorship access are often the thinnest, this program is built to fill exactly that gap.

Career switchers

If you're moving into software development from another field, the twelve-week structure gives you a defined, evidence-backed path instead of an open-ended search.

Self-learners who've hit a ceiling

You've done the free tutorials. You have a GitHub with a few repos. What you're missing is structure, review, and a project that actually stands out.

A laptop capable of running a modern code editor and local dev servers, stable internet, and basic logical reasoning: that's the entire prerequisite list.

OUTCOMES

Not "you'll learn React." This.

- Build modern, responsive, production-shaped web applications from scratch
- Design and ship RESTful APIs with real validation, error handling, and clean architecture
- Model and query a production-style database with MongoDB and Mongoose
- Implement authentication, authorization, and a real security-hardening pass
- Migrate a full application to Next.js and reason correctly about server vs. client rendering
- Test, containerize, and deploy an application through a working CI/CD pipeline
- Integrate a genuine GenAI-powered feature into a live product, safely and with validated output
- Read an unfamiliar codebase, open a Pull Request, and give real code review feedback
- Document and present a full-stack project the way a hiring manager expects to see it
- Walk into a technical, system-design, or behavioral interview round and hold your own

TOOLS & TECHNOLOGIES

Everything you'll have real, hands-on hours with.

LANGUAGES

JavaScript (ES6+)

TypeScript

FRONTEND

React

React Router

Next.js

Tailwind CSS

shadcn/ui

React Hook Form

Zod

Axios

TanStack Query

Redux Toolkit

BACKEND

Node.js

Express.js

JWT

bcrypt

Cloudinary

Socket.IO

DATABASE

MongoDB

Mongoose

MongoDB Compass

TESTING

Vitest

Testing Library

Playwright

DEVOPS & DEPLOYMENT

Docker

GitHub Actions

Vercel

Railway / Render

VERSION CONTROL & ENVIRONMENT

Git

GitHub

VS Code

Postman

GENAI TOOLS & APIS

ChatGPT

Claude

Cursor

GitHub Copilot

Gemini

v0

Bolt.new

Lovable

OpenAI API

Anthropic API

Every tool on this page is free-tier or student-license accessible. No paid software is required to complete the course.

The program, in one page.

12 Weeks

Program Duration

11 + 1

Milestone Projects +
Capstone

12

Saturday Career Sessions

1

Capstone, Presented Live to
a Panel

Live

100% Live, Mentor-Led
Learning

4

Mock Interview Rounds

4

Formal Code Review
Checkpoints

1

LUMORA Certificate of
Completion

LUMORA issues its own MERN Stack Development with GenAI Certificate of Completion on passing the course (a weighted score of 70% or higher, with the capstone submitted and presented). Students are also prepared for, and encouraged to pursue independently, the Meta Front-End and Back-End Developer Professional Certificates, freeCodeCamp's Full Stack Developer certification, and the MongoDB Certified Developer Associate credential.

Straight answers, before you apply.

Q. Do I need coding experience to join?

No. The program assumes zero prior experience with JavaScript, TypeScript, React, Node.js, Express, MongoDB, Next.js, or Git, and builds every concept from the ground up.

Q. What laptop do I need?

Any Windows, Mac, or Linux laptop capable of running a modern code editor and local development servers, with stable internet. No paid software is required for any part of the course.

Q. Will I actually build real projects?

Yes. Eleven milestone projects plus a capstone, each a genuinely different kind of application, every one deployed and pushed to GitHub with full documentation.

Q. Is AI actually taught, or just mentioned?

It's a thread through all twelve weeks, from Day 1's AI-tool onboarding to Week 12's dedicated unit on integrating a real LLM-powered feature into a production application.

Q. How are classes conducted?

Live: 60 weekday sessions (Monday to Friday, 1.5 hours each) plus 12 Saturday career sessions, all run with an instructor in real time, not pre-recorded.

Q. Will I get real mentorship and feedback?

Yes. Formal code reviews happen at Weeks 4, 7, 9, and 11; peer reviews at Weeks 4 and 9; and a 1:1 resume and portfolio review measured against real hiring-manager expectations.

Q. What do I walk away with?

A GitHub portfolio of 11 milestone projects plus a capstone, a reviewed resume and LinkedIn profile, four mock interview rounds behind you, and a LUMORA Certificate of Completion.

LUMORA SPACE

YOUR NEXT TWELVE WEEKS

Stop learning to code. Start becoming someone who ships it.

Twelve weeks, one cohort, eleven projects, one capstone with your name on it, and a clear answer, finally, to "what do I learn next."

[Apply to the Next Cohort](#)

LUMORA SPACE

EMAIL

support@lumoraspace.in

WEBSITE

lumoraspace.in

PROGRAM

MERN Stack
Development
with GenAI

LUMORA SPACE

Curriculum designed for industry readiness, not academic completeness.

22